

国家级精品课程
山东省一流本科课程



程序设计基础（C语言）

信息科学与工程学院

潘玉奇

第2章 C语言基础

- ◆ 2.1 C语言的发展历程
- ◆ 2.2 C程序的特点及开发环境
- ◆ 2.3 输入输出简单的数据信息
- ◆ 2.4 C语言的运算符
- ◆ 2.5 C语言的数据类型
- ◆ 2.6 类型转换
- ◆ 2.7 C语言的基本标识
- ◆ 2.8 格式化输入输出函数完整版
- ◆ 2.9 C语言的程序结构
- ◆ 2.10 编译预处理命令



2.1 C语言的发展历程

◆ C语言的产生及发展

ALGOL60 → CPL → BCPL → B → C → ANSI C → ISO C

◆ 常用的C编译程序

- **msvc**（微软开发的编译器）
- **gcc**（Linux操作系统使用的编译器）
- **clang**（苹果公司开发的编译器）

2.2 C程序的特点及开发环境

2.2.1 C程序的组成及特点

1、语言特点

- (1) 语言简洁、紧凑，使用方便、灵活
- (2) 运算符丰富
- (3) 数据结构丰富
- (4) 结构化的控制语句
- (5) 语法限制不太严格，程序设计自由度大
- (6) 允许直接访问物理地址，能进行位运算
- (7) 生成目标代码质量高，程序执行效率高
- (8) 程序可移植性好



2.2.1 C程序的组成及特点

2. 结构特点

- (1) C语言是**以函数为核心**的语言，函数是C程序的基本单位，C程序由一系列的函数构成.
- (2) 一个C程序**有且只有一个main函数**，main函数是整个程序的入口，也是程序正常终止的出口
- (3) 一个C程序一般包含一个main函数和若干个其他函数

2.2.2 C程序的风格

1. 源程序的编写风格

- (1) 严格采用阶梯层次组织程序代码。
- (2) 对复杂的条件判断，应尽量使用括号。
- (3) 变量的定义，尽量位于函数的开始位置。
- (4) 采用规范的格式定义和设计各种函数。
- (5) 尽量不使用goto语句。
- (6) 尽量减少全局变量的使用。

2.2.2 C程序的风格

2. 变量的命名规则

变量名=属性+类型+对象描述

匈牙利命名法:

一个变量名由三部分组成，首先是变量属性（如全局变量，静态变量等），然后是变量的数据类型，最后描述一般用变量的英文意思或英文意思的缩写。

例: unsigned int g_uSrcIP, g_uDstIP;

说明:

unsigned int —— 数据类型

g_ —— 表示全局变量

u —— 表示无符号整型（即unsigned int）

Src —— 表示源（即单词source）

Dst —— 表示目的（即单词destination）

IP —— 表示网络地址

2.2.2 C程序的风格

3. C程序的注释

- (1) **函数头的注释：**一般从功能、参数、返回值、主要思路、调用方法、日期等六个方面进行注释
- (2) **变量的注释：**说明变量在函数或程序中的作用及意义
- (3) **文件的注释：**在每一个源文件的开头部分加入注释来说明该文件的用途
- (4) **其他注释：**在各功能模块的主要部分之前添加注释；在分支、循环结构等处尽可能加以释，以增强程序可读性

2.2.2 C程序的风格

4. C程序的开发环境

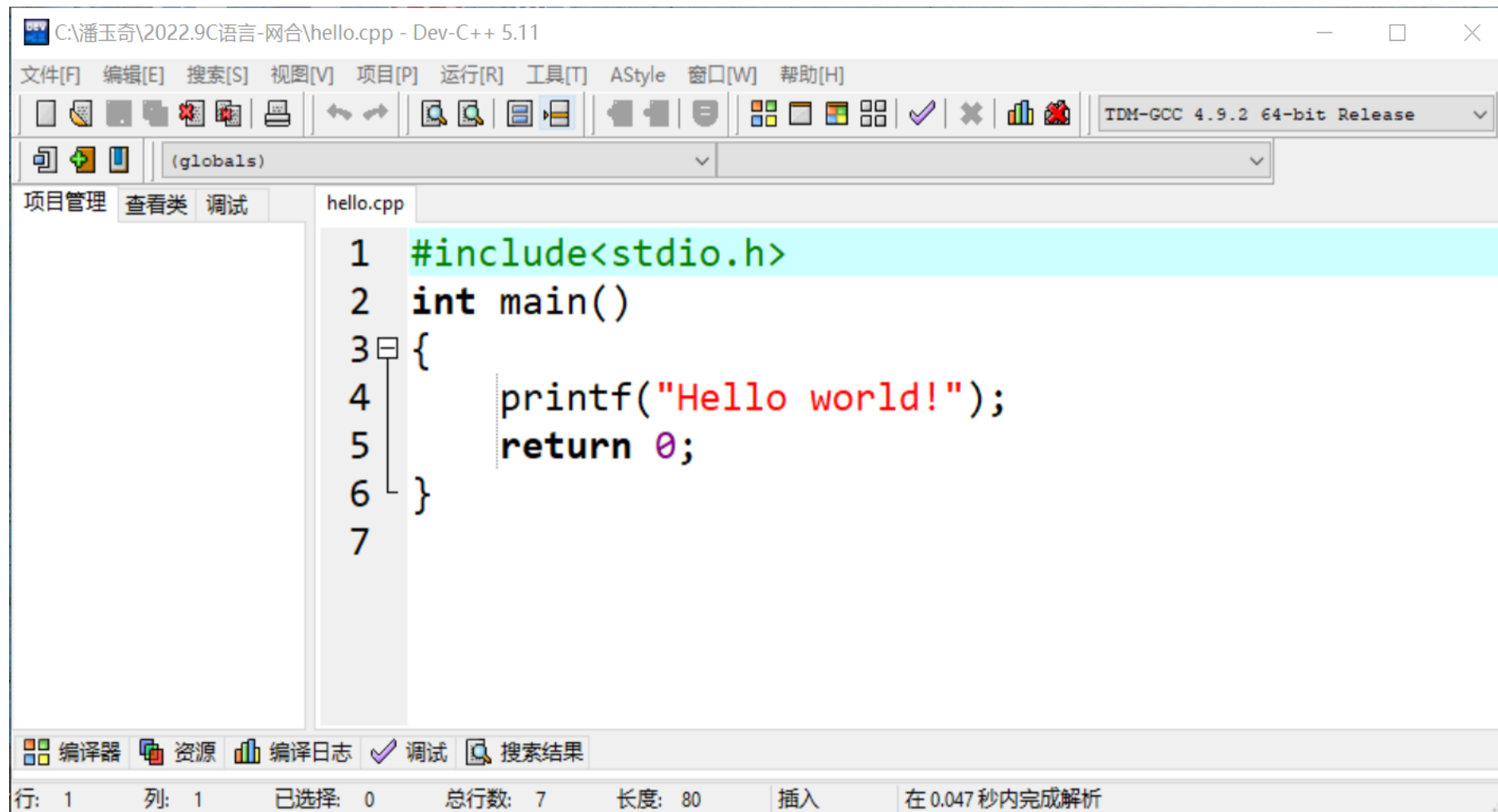
目前使用的大多数C编译系统都是集成环境(IDE)，把程序的编辑、编译、连接和运行全部集中在一个界面上进行。

目前，常用的C语言集成开发环境有：

- **Microsoft Visual C++**
- **Dev-C++**
- **C-Free**
- **Microsoft Visual Studio**
- **CodeBlocks**
- **QtCreator**

2.2.2 C程序的风格

◆ Dev-C++的环境



2.3 输入输出简单的数据信息

2.3.1 输出文本信息

【例2.1】一个最简单的C程序，在屏幕上显示“Hello world!”。这是一个大多数高级语言编程教材都使用的“网红”例题

```
#include<stdio.h>
```

文件包含命令,其功能是将头文件`stdio.h`的内容包含到用户当前的源程序中。

```
int main( )
```

每个C程序必须有`main`函数。`main`是函数名，`main`后的`圆括号`不能省略
`int`表示该函数的返回值为整型数据

```
{
```

```
    printf("Hello world!");
```

```
    return 0;
```

`printf`是标准输出函数,要调用它必须加`#include<stdio.h>`命令。`printf`函数的作用是将双引号中的内容输出到显示器上

```
}
```

表示`main`函数执行完后正常返回到操作系统

2.3.1 输出文本信息

➤ 除了输出英文，也可以输出中文

【例2.2】输出古诗《静夜思》

```
#include<stdio.h>
int main( )
{
    printf(" 静夜思\n");
    printf("\n");
    printf("床前明月光， \n");
    printf("疑是地上霜。 \n");
    printf("举头望明月， \n");
    printf("低头思故乡。 \n");
    return 0;
}
```

\n的作用是换行，在printf()中只写一个“\n”时，会输出一个空行。

静夜思

床前明月光，
疑是地上霜。
举头望明月，
低头思故乡。

2.3.1 输出文本信息

【练习2.1】请编程分别输出以下图形：

*****	*	*	*
*****	**	***	***
*****	***	*****	*****
*****	****	*****	***
*****	*****	*****	*

2.3.2 输出整数

➤ 问题：需要计算机输出以下信息：x=3，程序该怎么写？

方法1:

```
#include<stdio.h>
int main( )
{
    printf("x=3");
    return 0;
}
```

方法2:

```
#include<stdio.h>
int main( )
{
    printf("x=%d", 3);
    return 0;
}
```

%d是指这个位置要输出一个整数，具体的整数是什么要看输出项写的是是什么，在这里输出项就是逗号后面的整数**3**。

2.3.2 输出整数

【例2.3】计算机实现两个整数的加、减、乘、除。

假设这两个数是6和3，编程输出6和3的和、差、积、商。

```
#include<stdio.h>
int main()
{
    printf("6+3=%d\n", 6+3);
    printf("6-3=%d\n", 6-3);
    printf("6*3=%d\n", 6*3);
    printf("6/3=%d\n", 6/3);
    return 0;
}
```

输出：

6+3=9

6-3=3

6*3=18

6/3=2

说明：

printf()函数里的内容分为两个部分，一部分是双引号括起来的，另一部分是输出项，它们之间用逗号分隔。

2.3.3 格式化输出函数

◆ 格式化输出函数的一般格式:

printf(格式控制字符串, 输出项列表);

➤ 格式控制字符串是用一对双引号括起来的字符串, 用来指定输出数据项的类型和格式, 它包括两部分:

① **普通字符**: 即需要原样输出的字符, 包括转义字符。

② **格式说明**: 由“%”和**格式字符**组成, 如%d, %f等,

%d用来输出整数, %f用来输出单精度实数。

格式说明总是由“%”开始, 到格式字符终止,

它的作用是将数据项按指定的格式输出。

2.3.3 格式化输出函数

【例2.3】 代码

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    printf("6+3=%d\n", 6+3);
```

```
    printf("6-3=%d\n", 6-3);
```

```
    printf("6*3=%d\n", 6*3);
```

```
    printf("6/3=%d\n", 6/3);
```

```
    return 0;
```

```
}
```

➤ 问题：如果要计算20和4的和、差、积、商怎么办？

```
printf("20+4=%d\n", 20+4);
```

```
printf("20-4=%d\n", 20-4);
```

```
printf("20*4=%d\n", 20*4);
```

```
printf("20/4=%d\n", 20/4);
```

如果每换两个数就要修改源代码，这个程序未免也写得太low了。最好是每次执行程序都可以输入任意两个数进行计算。

为了达到这个目的，我们还需要学习输入函数，还需要知道什么是常量，什么是变量。

2.3.3 格式化输出函数

【例2.4】从键盘输入两个整数，计算这两数的和、差、积、商。

//代码1

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b;           //定义两个整型变量
```

```
    scanf("%d%d", &a, &b); //输入函数，从键盘输入两个整数
```

```
    printf("a+b=%d\n", a+b); //第1种输出格式
```

```
    printf("a-b=%d\n", a-b);
```

```
    printf("a*b=%d\n", a*b);
```

```
    printf("a/b=%d\n", a/b);
```

```
    return 0;
```

```
}
```

2.3.3 格式化输出函数

【例2.4】从键盘输入两个整数，计算这两数的和、差、积、商。

//代码2

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b;
```

```
    scanf("%d%d", &a, &b);
```

```
    printf("%d+%d=%d\n", a, b, a+b);    //第2种输出格式
```

```
    printf(" %d- %d=%d\n", a, b, a-b);
```

```
    printf(" %d*%d=%d\n", a, b, a*b);
```

```
    printf(" %d/%d=%d\n", a, b, a/b);
```

```
    return 0;
```

```
}
```

2.3.3 格式化输出函数

【例2.4】从键盘输入两个整数，计算这两数的和、差、积、商。

//代码3

```
#include<stdio.h>
```

```
int main()
```

```
{ int a, b;
```

```
int r1, r2, r3, r4; // r1—r4分别存放和、差、积、商的计算结果
```

```
scanf("%d%d", &a, &b);
```

```
r1=a+b;    r2=a-b;    r3=a*b;    r4=a/b;
```

```
printf(" %d+%d=%d\n", a, b, r1); //第3种输出格式
```

```
printf(" %d-%d=%d\n", a, b, r2);
```

```
printf(" %d*%d=%d\n", a, b, r3);
```

```
printf(" %d/%d=%d\n", a, b, r4);
```

```
return 0;
```

```
}
```

2.3.4 常量和变量

◆ **常量(constant)**: 在程序执行期间值不发生变化的量

(1) 直接常量:

整型常量: 15、0、-6

实型常量: 2.4、0.0、-3.78

字符常量: 'a'、'#'、'*'、'\n'

(2) **符号常量**: 在程序中用标识符代表的常数。

定义符号常量的格式:

#define 标识符 常数

● 使用符号常量的优点

- ✓ 含义清楚
- ✓ 修改方便

```
#include <stdio.h>
```

```
#define PI 3.14159
```

```
int main( )
```

```
{
```

```
float r, c, s;
```

```
r=3.5;
```

```
c=2*PI*r;
```

```
s=PI*r*r;
```

```
printf("%f %f", c, s);
```

```
return 0;
```

```
}
```

符号常量

直接常量

2.3.4 常量和变量

(3) 用关键字**const**定义常量（补充内容）

```
#include <stdio.h>
```

```
#define PI 3.14
```

```
int main( )
```

```
{
```

```
float r, c, s;
```

```
r=3.5;
```

```
c=2*PI*r;
```

```
s=PI*r*r;
```

```
printf("%f, %f", c, s);
```

```
return 0;
```

```
}
```

```
#include <stdio.h>
```

```
int main( )
```

```
{
```

```
const float PI=3.1415926;
```

```
float r, c, s;
```

```
scanf("%f", &r);
```

```
c=2*PI*r;
```

```
s=PI*r*r;
```

```
printf("%f, %f", c, s);
```

```
return 0;
```

```
}
```

2.3.4 常量和变量

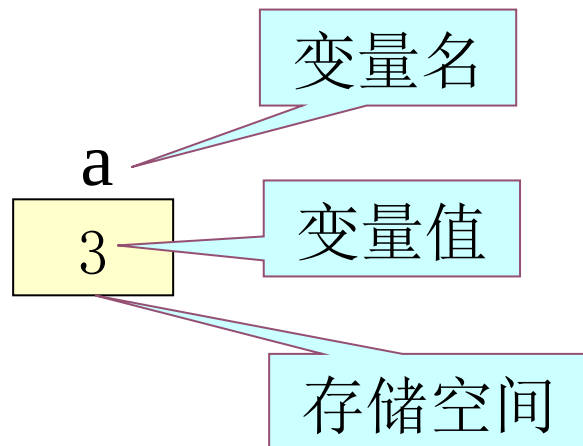
◆ **变量(variable):** 在程序执行期间值可以变化的量

变量的定义格式:

数据类型 **变量名列表;**

例: **int a, b;**

float r, c, s;



➤ **变量说明:**

- ① 进行变量声明后, 计算机系统会为声明的变量分配存储空间, 用以存放数据。
- ② 变量的存储空间可能由一个或多个字节组成, 内存中的每个字节都有自己的地址, 变量名实际上是一个符号地址。在程序中对变量的赋值和取值操作实际上是通过变量名找到相应的内存地址, 然后从对应的存储空间中读取数据。

2.3.4 常量和变量

➤ 变量说明:

③ C语言中的变量必须“先定义，后使用”。

```
int a, b;
```

```
a=3; b=5;
```

```
c=a+b;
```

c 没有定义就直接使用，是错误的

④ 变量的初始化，在定义变量的同时，给变量一个初始数据。未初始化的变量的值是不确定的，赋值之前请不要使用其进行计算。

例:

```
int a=3, b, c; //变量a进行了初始化，b和c没有初始化
```

```
c=b+1; //无法确定c的值，因为不知道b的值是多少
```

```
c=a+1; // c的值为4
```


2.3.5 格式化输入函数

◆ 格式化输入函数的一般格式:

scanf (格式控制字符串, 地址列表);

说明:

该函数的功能是按照用户指定的格式, 由键盘输入若干个数据

(1) 格式控制字符串的含义与printf类似, 它指定输入数据项的类型和格式。

例:

```
int a, b;
```

```
scanf("%d%d", &a, &b); // 注意, 输入整型变量用%d
```

```
float r, c, s;
```

```
scanf("%f", &r); // 注意, 输入单精度实型变量用%f
```

2.3.5 格式化输入函数

◆ 格式化输入函数的一般格式:

scanf (格式控制字符串, 地址列表);

说明:

(2) 地址列表是由若干个地址组成的列表, 可以是变量的地址, 即: **&变量名**, (C语言中&称为地址运算符)或字符串的首地址 (将在第4章字符数组中介绍。

例:

```
int a, b;
```

```
scanf("%d%d", &a, &b);
```

把变量a和b的内存地址告诉scanf()函数, 系统就会把输入的数据存储在对应的变量中。

2.3.6 简单程序设计

◆ 小试牛刀

【练习2.2】输入三个整数，计算并输出它们的和与积。

（输出结果分2行，第1行为3个数的和，第2行为3个数的积）

测试数据：

输入：3 4 5✓

输出：12

60

【练习2.3】小明今天想吃水果，他准备买2斤苹果，3斤桃子，从键盘输入1斤苹果和1斤桃子的价格，计算并输出小明买水果的开销。

测试数据：

输入：6 4✓

输出：24

2.4 C语言的运算符

2.4.1 简单赋值运算符

- ◆ 赋值运算符：“=”
- ◆ 优先级很低，仅高于逗号运算符
- ◆ 赋值运算符是右结合性
- ◆ 赋值表达式的一般形式：
变量 = 表达式
- ◆ 赋值表达式的计算规则：
先计算赋值运算符“=”右边表达式的值，
然后将该值赋给左边的变量。

注意：赋值运算符的左侧必须是变量

```
#include <stdio.h>
#define PI 3.14
int main( )
{ float r, c, s;
  r = 3.5;
  c = 2*PI*r;
  s = PI*r*r;
  printf("%f, %f", c, s);
  return 0;
}
```

赋值运算符

2.4.1 简单赋值运算符

◆ 变量赋值的特点

- ① 变量必须**先定义，后使用**。
- ② 变量未被赋值前，值不确定。
- ③ 对变量**赋值过程是“覆盖”过程**，用新值去替换旧值。
- ④ 计算表达式时，只是读出变量的值去参加计算，
所有变量都保持原来的值不变。

2.4.1 简单赋值运算符

【例】变量赋值举例

```
#include<stdio.h>
```

```
int main( )
```

```
{  int  a, b, c;
```

编译时出错，因x没有定义

```
  x=15;
```

```
  c=a+10;
```

因a的值不确定(随机值)，这种运算没意义

```
  printf("a=%d, c=%d\n", a, c);
```

```
  a=15;
```

赋值语句，将常数15赋值给变量a

```
  b=20;
```

```
  c=a+b;
```

赋值语句,先计算15+20的结果,将35赋值给变量c

```
  printf("a=%d, b=%d, c=%d\n", a, b, c);
```

```
  c=10;
```

将10赋值给变量c,即用10覆盖了原来的35

```
  a=b=c;
```

注意赋值运算的右结合性,先将变量c的值赋给变量b,再将变量b的值赋给变量a,最后a, b, c的值都是10

```
  return 0;
```

```
}
```

2.4.1 简单赋值运算符

【练习2.4】已知a=135， b=256， 试交换两个变量的值。

提示：交换两个变量的值，就好像有2个瓶子，a瓶装酱油，b瓶装醋，将酱油和醋互换，即a瓶装醋，b瓶装酱油。

```
#include<stdio.h>
```

```
int main()
```

```
{  int a, b, c;    // 定义3个整型变量(酱油瓶a, 醋瓶b, 空瓶c)
```

```
    a=135;        // 变量a赋值(瓶a倒入酱油)
```

```
    b=256;        // 变量b赋值(瓶b倒入醋)
```

```
    printf("a=%d, b=%d\n");
```

```
    c=a;        // 将变量a的数据赋值给变量c(酱油倒入空瓶c)
```

```
    a=b;        // 将变量b的数据赋值给变量a(醋倒入酱油瓶a)
```

```
    b=c;        // 将变量c的数据赋值给变量b(酱油倒入醋瓶b)
```

```
    printf("a=%d, b=%d\n");
```

```
    return 0;
```

```
}
```

2.4.2 基本算术运算符

基本算术运算符包括：加(+)、减(-)、乘(*)、除(/)、取余(%)，它们是双目运算符，左结合性。

1. 除法运算

如果两个运算对象都是**整型**数据，则结果也是**整型**数据（即取整，舍去小数部分）。若想结果是**实型**数据，则两个运算对象中**必须有一个是实型**数据。

例： $1/4=0$ $1/5.0=0.2$

$5/2=2$ $2.0/4=0.5$

2.4.2 基本算术运算符

【例2.6】输入三角形的底和高，计算三角形面积并输出。

三角形面积公式： $area = \frac{1}{2} * 底 * 高$

提示：用float定义变量

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
float a, h, area; //变量a表示底，变量h表示高，变量area表示面积
```

```
scanf("%f%f", &a, &h); //输入底和高
```

```
area=1/2*a*h; //计算三角形面积
```

```
printf("%f\n", area); //输出面积
```

```
return 0;
```

```
}
```

程序的输出结果都是0.000000
为什么会这样呢？

这里1/2的结果不是我们以为的0.5，而是0
注意：除号两边都是整数时是整除

2.4.2 基本算术运算符

2. 取余运算

取余运算要求两个运算对象都必须是**整型**数据，结果也是**整型**数据，且余数的符号与被除数的符号相同。

例： $12\%5=2$ $12\%(-5)=2$

$3\%5=3$ $(-12)\%5=-2$

2.4.2 基本算术运算符

【例2.7】输入一个大于100的整数，对这个数进行处理，输出一个表示月份的整数。

分析：

假设整数为 x ，用 $x\%12$ 对不对？
还是应该用 $x\%13$ ？

如果用 $x\%12$ ，能得到的数的范围是0~11，多了个零，少了个12。

如果用 $x\%13$ ，能得到的数的范围是0~12，也是多了个零。

```
#include<stdio.h>
int main()
{
    int x;
    scanf("%d", &x);
    x=x%12+1;

    printf("%d\n", x);
    return 0;
}
```

2.4.2 基本算术运算符

【练习】由键盘输入一个10~99之间的整数，将该数进行数位分解，分别输出其个位数字和十位数字（两数间用逗号分隔）。

例如，输入85，输出：5，8。 提示：用整除和取余运算实现

```
#include<stdio.h>
```

```
int main()
```

```
{  int x, a, b;    //变量a存放个位数字，b存放十位数字
```

```
    printf("输入一个10~99之间的整数："); //提示用户的信息
```

```
    // 在ACM(OJ)或CG平台做作业时一般是不加的
```

```
    scanf("%d", &x);
```

```
    a=x%10;
```

```
    b=x/10;
```

```
    printf("%d, %d \n", a, b);
```

```
    return 0;
```

```
}
```

2.4.3 复合算术赋值运算符

在赋值运算符“=”前加上算术运算符可以构成复合算术赋值运算符，右结合性，包括： $+=$ ， $-=$ ， $*=$ ， $/=$ ， $\%=$ 。

例： $x=x+1;$ $\rightarrow$ $x+=1;$
 $sum=sum+n;$ $\rightarrow$ $sum+=n;$

例：

$int\ a=1, b=2, c=3;$
 $a+=3;$ *//相当于计算 $a=a+3;$*
 $b*=a;$ *//相当于计算 $b=b*a;$*

说明：

计算方式是先计算右边表达式的值，再将此值与左边的变量进行运算，其结果存入左边的变量中

例： $a*=b+4;$

还原为： $a=a*b+4;$ 

或是： $a=a*(b+4);$ 

注意：右边表达式是一个整体，将复合赋值表达式还原时，应加上括号避免出错

2.4.4 自加、自减运算符

自加、自减运算符是怎么产生的呢？

例：

```
int a=1, b=10;
```

```
a+=1;    //等价于 a=a+1;
```

```
b-=1;    //等价于 b=b-1;
```

为了使代码少一点再少一点，
可以将a+=1; 简写成a++; 或 ++a;
同理， b-=1; 可简写成b--; 或 --b;

自加(++)自减(--)是单目运算符，右结合性。

注意：运算对象只能是变量，不能是常量或表达式。

例：

```
int a=1, b=2;
```

```
a++;    //正确，变量可以进行自加、自减
```

```
(a+b)++; //错误，表达式不能进行自加、自减
```

```
2++;    //错误，常量不能进行自加、自减
```

2.4.4 自加、自减运算符

◆ 自加、自减运算符的前缀/后缀形式

$i=i+1$; $\rightarrow$ $i+=1$; $\rightarrow$ $i++$; 或 $++i$;

$i=i-1$; $\rightarrow$ $i-=1$; $\rightarrow$ $i--$; 或 $--i$;

➤ 当自加、自减运算符单独使用时，前缀、后缀形式并没有什么区别，都是变量的值自加1或自减1

➤ 当自加、自减运算符出现在表达式中，前缀和后缀形式的计算结果就完全不同了。

(1) 后缀形式：先使用*i*的当前值，再使*i*的值加1(减1)

(2) 前缀形式：先对*i*加1(减1)，再使用*i*变化后的值

例：

int i=3, j=6;

int a, b;

j++;

++j;

a=i++;

b=++i;

j=7

j=8

a=3, i=4

i=5, b=5

2.4.4 自加、自减运算符

【例2.8】 自加、自减运算的应用

```
#include<stdio.h>
```

```
int main()
```

```
{ int a=1, b=1, c, d, x, y;
```

```
    c=++a;
```

```
    d=b++;
```

```
    printf("c=%d, d=%d\n", c, d);
```

```
    printf("a=%d, b=%d\n", a, b);
```

```
    x=(a++)+(a++);
```

```
    printf("x=%d, a=%d\n", x, a);
```

```
    y=(++b)+(++b);
```

```
    printf("y=%d, b=%d\n", y, b);
```

```
    printf("%d, %d\n", x++, ++y);
```

```
    printf("x=%d, y=%d\n", x, y);
```

```
    return 0;
```

```
}
```

程序输出:

c=2, d=1

a=2, b=2

x=4, a=4

y=8, b=4

4, 9

x=5, y=9

先将a的原值2取出进行a+a加法计算，得到结果4赋给变量x，然后a再进行两次加1的计算，最后a的值为4

变量b先进行2次加1计算，b的值变为4，然后再计算b+b，即4+4=8，最后将结果8赋给变量y

因x原值为4，y原值为8，输出时先输出x的原值4，然后x再自加1变为5，而y要先加1变为9，再输出y的值9。

2.4.4 自加、自减运算符

【练习】写出程序运行结果

```
#include<stdio.h>
```

```
int main ( )
```

```
{  int a, b, c, d;
```

```
    int i=3, j=5, k=3;
```

```
    a=(i++)*j;
```

```
    b=(++i)*j;
```

```
    c= - k++;
```

```
    d= - ++k;
```

```
    printf("a=%d, b=%d\n", a, b);
```

```
    printf("c=%d, d=%d\n", c, d);
```

```
    return 0;
```

```
}
```

输出:

a=15, b=25

c=-3, d=-5

a=3*5=15 i=4

i=5 b=5*5=25

c= - (k++); c=-3 k=4

d= - (++k); k=5 d=-5

2.4.5 逗号运算符

- ◆ 逗号运算符也称顺序求值运算符，**优先级最低，左结合性。**
- ◆ **逗号表达式的一般形式：**

表达式1， 表达式2， …， 表达式n

- ◆ **计算过程：** 从左至右依次计算各表达式的值，最后一个表达式n的值即为逗号式的值。

例：

int a, b, c, x, y; //这里的逗号是分隔符，不是逗号运算符

a=1, b=2, c=3; //逗号表达式，给a,b,c依次赋值，逗号表达式的值为3

a++, b+4, c-2; //逗号表达式的值为1

//注意计算后a值为2，b值还是2，c值还是3

2.4.5 逗号运算符

注意: 逗号运算符的优先级最低

$x=(a=3, 6*3);$ 与

$x=a=3, 6*3;$ 是不同的

① $a=3$

② $6*3$ 得 18, 逗号表达式的值为18, $x=18$

① $a=3$ $x=3$

② $6*3$ 得 18, 逗号表达式的值为18, 但 x 的值为 3

【练习】写出程序运行结果。

```
#include <stdio.h>
```

```
int main ( )
```

```
{ int a, b, c, d;
```

```
   $a=(c=100, d=200, c+d);$ 
```

```
   $b=(c=d=0, c+50);$ 
```

```
  printf(" %d, %d, %d, %d\n ", a, b, c, d);
```

```
  return 0;
```

```
}
```

输出:

300, 50, 0, 0

2.4.6 C语言的运算符和表达式

◆ 运算符可以由一个或多个字符组成，表示各种运算。

◆ 根据参与运算的操作数的个数运算符可分为：

单目、双目、三目运算符。

(1) 算术运算符（+、-、*、/、%、++、--）

(2) 关系运算符（>、<、==、>=、<=、!=）

(3) 逻辑运算符（&&、||、!）

(4) 位运算符（<<、>>、~、&、|、^）

(5) 赋值运算符（=、+=、-=、*=、/=等扩展赋值运算符）

(6) 条件运算符（?:）

(7) 逗号运算符（,）

(8) 指针运算符（*、&）

(9) 求字节数运算符（sizeof）

(10) 特殊运算符（.、->、()、[]、(类型)）

2.4.6 C语言的运算符和表达式

- ◆ 表达式是由常量、变量、函数等通过运算符连接起来而形成的一个有意义的算式
- ◆ 一个表达式代表着一个具有特定数据类型的具体值
- ◆ C语言提供的表达式类型：

算术表达式，如： $3+4*5$

赋值表达式，如： $a=3$

关系表达式，如： $a>b$ 、 $x!=y$

逻辑表达式，如： $x!=y \&\&a>b$

条件表达式，如： $a>b?a:b$

逗号表达式，如： $a=3,b=4,c=5$

位表达式，如： $a<<2$

指针表达式，如： $p->2$ 、 $\&a$

2.5 C语言的数据类型

◆ 对于整数类型，你目前都掌握的内容

- ① 在程序中直接使用整数常量和整数表达式
- ② 在程序中用`int` 声明整型变量，例如 `int a, b;` 。
- ③ 输入输出整数或整型变量时，在格式字符串中要用`%d`,
例如 `printf("%d\n", 6+3);` 。
- ④ 两个整数做除法时，结果是整数，例如 $5/2=2$, $1/2=0$ 。
- ⑤ 两个整数做取余运算，余数的符号与被除数的符号相同，
例如 $5\%2=1$, $(-5)\%3=-2$ 。
- ⑥ 整型变量可以做自加自减运算，例如 `int a=1; a++;`。

2.5 C语言的数据类型

◆ 对于整数类型，以下内容你会吗？

- 整数的二进制、八进制、十六进制怎么表示？
- 不同进制的整数如何相互转换？
- 整数在计算机内是如何存储的？
- 正整数怎么存储？负整数又怎么存储？
- int型整数能表示的范围大小是多少？
- 除了int型，还有其他的整数类型吗？

2.5.1 整数类型

1. 整型常量

C语言中的整型（integer）常量即整常数，通常采用十进制、八进制和十六进制表示。

(1) 十进制整数 56 , -23 , 0

(2) 八进制整数 以零开头 034 , 012

(3) 十六进制整数 以0x开头 0x28 , 0x1fa9

十进制	1	2	3	4	5	6	7	8
八进制	1	2	3	4	5	6	7	10
十六进制	1	2	3	4	5	6	7	8
二进制	0001	0010	0011	0100	0101	0110	0111	1000

十进制	9	10	11	12	13	14	15	16
八进制	11	12	13	14	15	16	17	20
十六进制	9	A	B	C	D	E	F	10
二进制	1001	1010	1011	1100	1101	1110	1111	10000

2.5.1 整数类型

【例2.9】用不同的进制形式输出一个整数。

```
#include <stdio.h>
```

```
int main ( )
```

```
{   int a;
```

```
    a=100;
```

```
    printf("a=%d\n", a);
```

```
    printf("a=%o\n", a);
```

```
    printf("a=%x\n", a);
```

```
    return 0;
```

```
}
```

// d代表10进制 “decimal”

// o代表8进制 “octal”

// x代表16进制 “hexadecimal”

输出结果:

a=100

a=144

a=64

2.5.1 整数类型

【例2.10】 程序中直接使用八进制和十六进制整数。

```
#include<stdio.h>
```

```
int main()
```

```
{ int a, b;
```

```
    a=0100;           // 变量a赋值为一个8进制整数
```

```
    printf("a=%d\n", a);
```

```
    printf("a=%o\n", a);
```

```
    printf("a=%x\n", a);
```

```
    b=0x100;         // 变量b赋值为一个16进制整数
```

```
    printf("b=%d\n", b);
```

```
    printf("b=%o\n", b);
```

```
    printf("b=%x\n", b);
```

```
    return 0;
```

```
}
```

输出结果:

a=64

a=100

a=40

b=256

b=400

b=100

2.5.1 整数类型

2. 不同进制整数的转换方法

(1) 2进制整数转换为10进制整数的方法：

按权展开逐个相加的方法。

$$(1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 2 + 1 = (11)_{10}$$

(2) 10进制整数转换为2进制整数的方法：

除2取余法。

$$(25)_{10} = (11001)_2$$

2		25	余 1
2		12	余 0
2		6	余 0
2		3	余 1
2		1	余 1
		0	

逆向排列

2.5.1 整数类型

2. 不同进制整数的转换方法

(3) 8进制整数转换为10进制整数的方法：

按权展开逐个相加的方法。

$$(164)_8 = 1 \times 8^2 + 6 \times 8^1 + 4 \times 8^0 = 64 + 48 + 4 = (116)_{10}$$

(4) 10进制整数转换为8进制整数的方法：

除8取余法。

$$(118)_{10} = (166)_8$$

$$8 \overline{) 118}$$

余 6

$$8 \overline{) 14}$$

余 6

$$8 \overline{) 1}$$

余 1

0

逆向排列

【练习】 $(100)_{16} = (?)_{10}$

$(200)_{10} = (?)_{16}$

2.5.1 整数类型

3. 整型变量及其内存分配

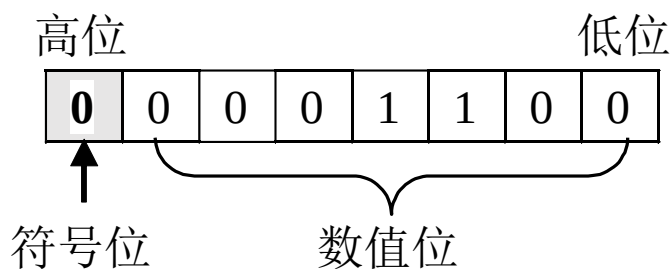
类型	类型说明符	字节	比特	数值范围
短整型	short	2	16	-32768 ~ 32767 即 $-2^{15} \sim (2^{15}-1)$
基本整型	int	4	32	-2147483648 ~ 2147483647 即 $-2^{31} \sim (2^{31}-1)$
长整型	long	4	32	-2147483648 ~ 2147483647 即 $-2^{31} \sim (2^{31}-1)$
超长整型	long long	8	64	-9223372036854775808 ~ 9223372036854775807 即 $-2^{63} \sim (2^{63}-1)$
无符号短整型	unsigned short	2	16	0 ~ 65535 即 $0 \sim (2^{16}-1)$
无符号基本整型	unsigned	4	32	0 ~ 4294967295 即 $0 \sim (2^{32}-1)$
无符号长整型	unsigned long	4	32	0 ~ 4294967295 即 $0 \sim (2^{32}-1)$
无符号超长整型	unsigned long long	8	64	0~18446744073709551615 即 $0 \sim (2^{64}-1)$

2.5.1 整数类型

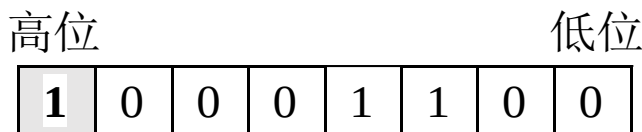
4. 整数的编码方式

- 整数X的原码：符号位的0或1表示X的正或负，数值位是X绝对值的二进制数

整数12的原码：



整数-12的原码：



$$12 + (-12) = 0$$

数0的原码是不唯一的，会有“正零”和“负零”之分

0000 0000

+0

1000 0000

-0

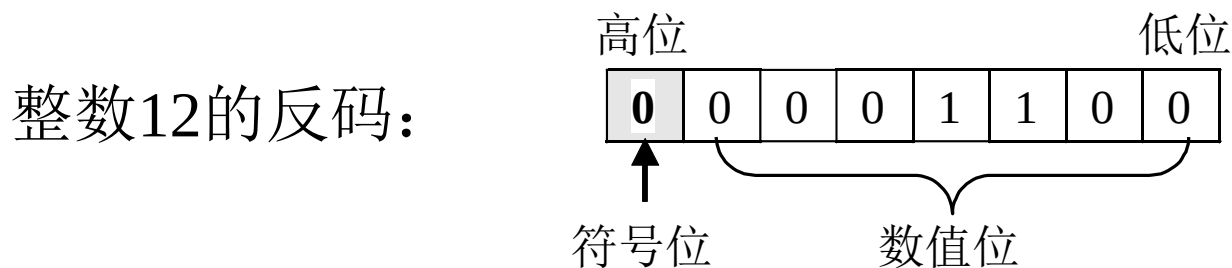
	0000	1100
+	1000	1100
	1001	1000



2.5.1 整数类型

4. 整数的编码方式

- **整数X的反码**：正数的表示方法与原码相同，负数的反码是把其原码除符号位以外的各数值位按位取反（即0变1，1变0）



数0的反码也是不唯一的，会有“正零”和“负零”之分。

0000 0000

+0

1111 1111

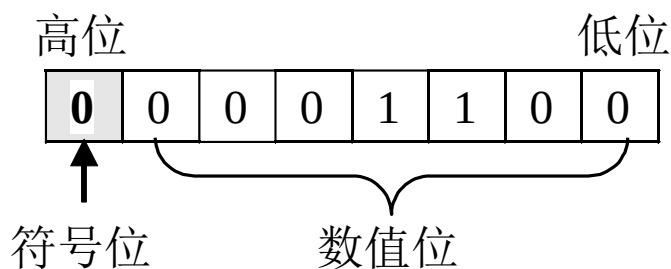
-0

2.5.1 整数类型

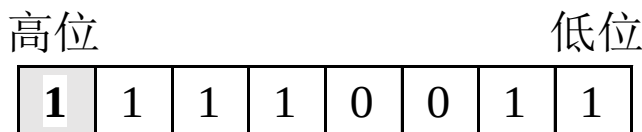
4. 整数的编码方式

- **整数X的补码**：正数的表示方法与原码相同，负数的补码是在其反码的最低位上加1。

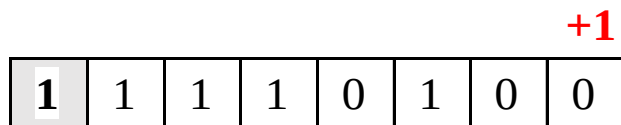
整数12的补码：



整数-12的反码：



整数-12的补码：



注意：数0的补码是唯一的。

2.5.1 整数类型

4. 整数的编码方式

➤ 整数类型的取值范围

(1) 带符号整数

short型的正数最大值为 32767 →

0111 1111	1111 1111
-----------	-----------

(计算 $2^{14}+2^{13}+\dots+2^2+2^1+2^0=16384+8192+\dots+4+2+1=32767$)

short型的负数的最小值为 -32768 →

1000 0000	0000 0000
-----------	-----------

(2) 无符号整数：最高位与其他位一起表示数值，只能存放正数
它与同样长度带符号整数相比，所能表示的正数值扩大一倍
范围是：0000 0000 0000 0000 ---- 1111 1111 1111 1111
即 0 ---- 65535 ($2^{15}+2^{14}+\dots+2^1+2^0$)

2.5.1 整数类型

5. 整形数据的溢出

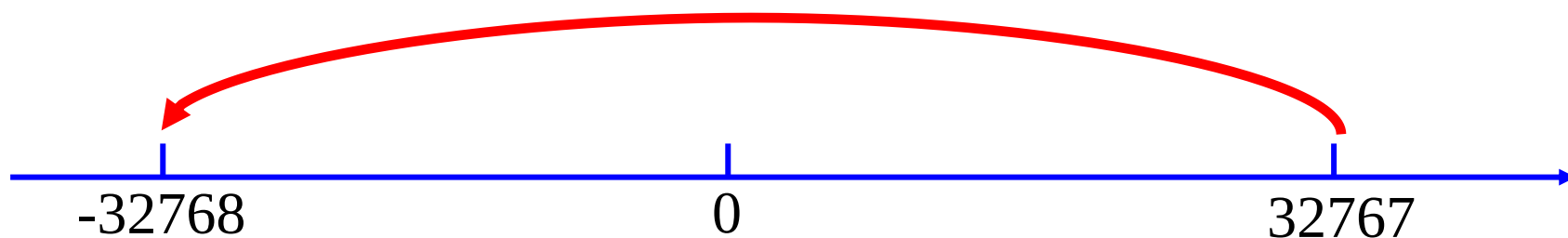
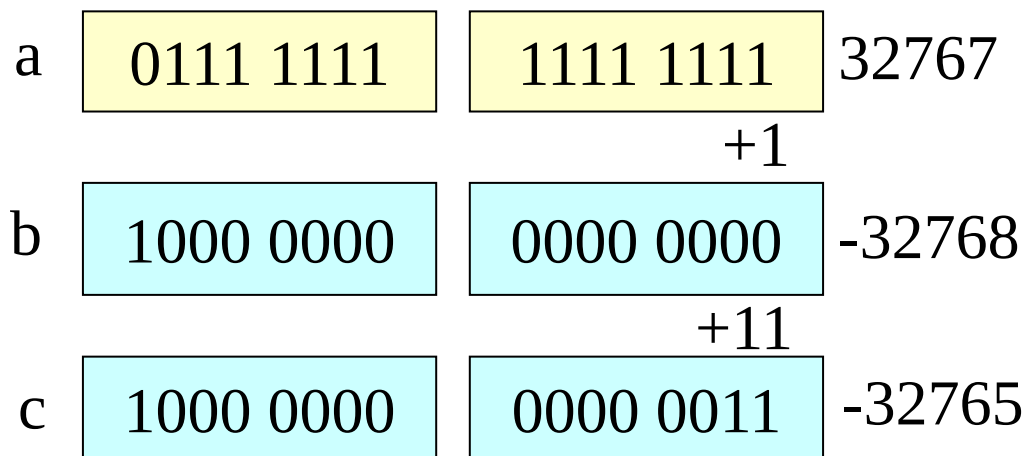
由于整数在内存里所占用的比特数是固定长度的，因此它能存储的最大值也是固定的，当存储的数据大于这个最大值时，将会导致整数的溢出。

例: `short a, b, c;`

`a=32767;`

`b=a+1;`

`c=b+3;`



说明: **Dev-C++5.11**是这样的, 其他C编译器不一定

2.5.2 实数类型

1. 实型常量

C语言中的实型（**real**）也称浮点型（**floating-point**），主要有以下两种表示形式。

(1) 十进制小数形式：由数码0~9和小数点组成。

例：0.0， 5.789， .13， 5.， -26.83。

注意：小数点必须有。

(2) 指数形式：由十进制小数(尾数)，加e或E及指数(阶码)组成

例：1.23e2， -5.78E3， 0.23E-2

注意：E(或e)的前、后必须有数字，且E后的指数只能是整数。

注意：一个实数的指数形式是不唯一的。

例：3.14的指数形式：3.14e0, 0.314E 1, 31.4E-1

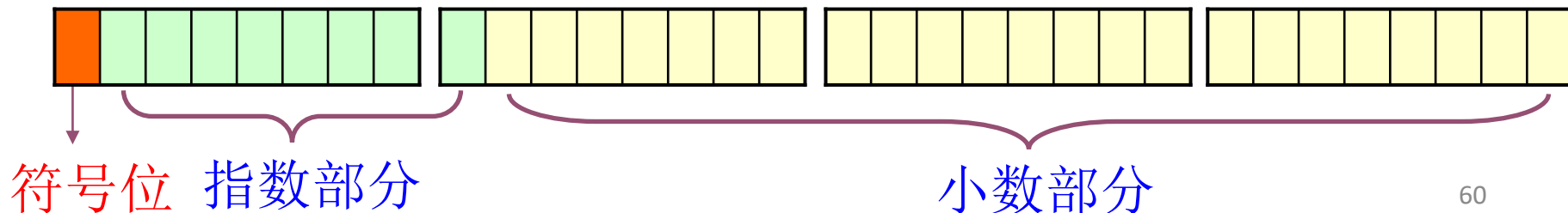
2.5.2 实数类型

2. 实型变量及其内存存储

实型变量分为单精度型、双精度型和长双精度型，
ISO C标准定义的实数类型如下表所示。

类型	类型说明符	字节数	比特数	有效数字	数值范围
单精度型	float	4	32	6-7	$-3.4 \times 10^{+38} \sim 3.4 \times 10^{+38}$
双精度型	double	8	64	15-16	$-1.7 \times 10^{+308} \sim 1.7 \times 10^{+308}$
长双精度型	long double	8	64	15-16	$-1.7 \times 10^{+308} \sim 1.7 \times 10^{+308}$

实型数据在内存中按指数形式存储，系统把数据分成小数部分和指数部分，在常用C编译系统下，以float型数据为例，用1位表示符号，用8位表示指数部分，用23位表示小数部分



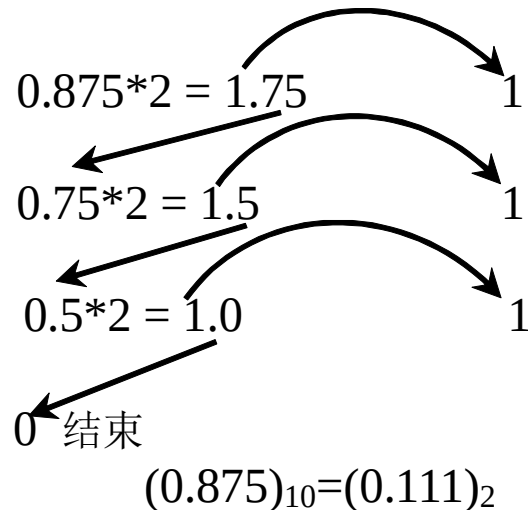
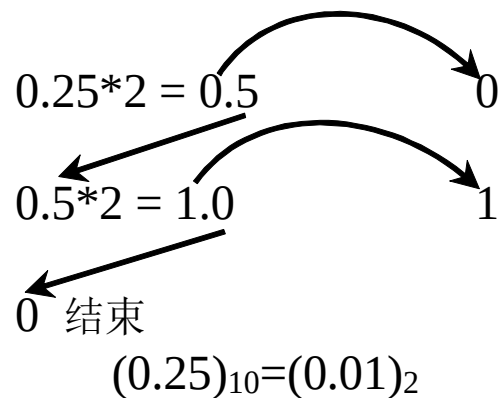
2.5.2 实数类型

2. 实型变量及其内存存储

十进制浮点数转换为二进制数的方法是以小数点为界，对整数部分和小数部分分别进行转换。

整数部分的转换方法是“除2取余法”，

小数部分的转换方法可以称为“乘2取整法”。



2.5.2 实数类型

3. 实型数据的误差

将10进制的小数转换为2进制形式时，多数情况下得到的是一个无限循环的二进制序列，由实型数据的存储方式可知，尾数部分的比特位是固定大小的，因此只能存储这个无限二进制序列的一部分，所以会存在一定的误差。

另外由于不同类型的实型数据在计算机中有效数字的位数不同，也导致了不能精确表示某个实数。

2.5.2 实数类型

【例2.14】实型数据的有效数字问题。

```
#include <stdio.h>
```

```
int main()
```

```
{   float a;
```

```
    double b;
```

```
    a=34567.333333;
```

```
    b=123456789123.66666666;
```

```
    printf("a=%f\nb=%lf\n", a, b);
```

```
    return 0;
```

```
}
```

输出结果：

a=34567.332031

b=123456789123.666670

说明：

因float型最多提供7位有效数字，变量a的整数部分已占5位，故小数点后只有前2位是有效数字，后4位均为无效数字。double型最多提供16位有效数字，变量b的整数部分占12位，所以小数点后只有前4位是有效数字。

2.5.2 实数类型

【练习2.6】 输入圆柱体的半径和高，编程计算圆柱体的表面积。
测试数据如下。

输入： 2 3

输出： area=62.799999

【练习2.7】 输入一个华氏温度F，计算对应的摄氏温度C并输出。

温度转换公式和测试数据如下。 $C = \frac{5}{9}(F - 32)$

输入： 100

输出： 37.777779

【练习2.8】 输入一个摄氏温度C，计算对应的华氏温度F并输出。

测试数据如下。

输入： 28

输出： 82.400002

2.5.3 字符类型

1. 字符常量

用单引号括起来的一个字符，或用单引号括起来的由反斜杠引导的转义字符。

例如：'a'，'A'，'#'，':', '\n' 。

说明：

- 'a' 和'A'是两个不同的字符
- 单引号内不能是单引号或反斜杠
- 转义字符。如：'\n', '\\' 表示一个反斜杠, '\'' 表示单引号

2.5.3 字符类型

常用转义字符及其含义

转义字符 ↵	含 义 ↵	ASCII 值 ↵
\0 ↵	字符串结束标志符 ↵	0 ↵ (00000000) ↵
\n ↵	换行，将当前位置移到下一行开头 ↵	10 ↵ (00001010) ↵
\t ↵	水平制表（跳到下个 Tab 的位置） ↵	9 ↵ (00001001) ↵
\b ↵	左退一格 ↵	8 ↵ (00001000) ↵
\r ↵	回车，将当前位置移到本行开头 ↵	13 ↵ (00001101) ↵
\a ↵	响铃 ↵	7 ↵ (00000111) ↵
\' ↵	单引号 ↵	39 ↵ (00100111) ↵
\" ↵	双引号 ↵	34 ↵ (00100010) ↵
\\ ↵	反斜杠 \ ↵	92 ↵ (01011100) ↵
\ddd ↵	1 到 3 位八进制数代表的字符 ↵	↵
\xhh ↵	1 到 2 位十六进制所代表的字符 ↵	↵

2.5.3 字符类型

2. 字符变量

字符变量用来存放一个字符，每个字符在内存中占用1个字节的存储空间，内存中实际存放的是字符对应的ASCII码。

例：char ch;

ch='A';

字母A的ASCII码为65,内存中存放为

0	1	0	0	0	0	0	1
---	---	---	---	---	---	---	---

注意：

在VS2013中，char型是有符号的，取值范围是-128~127。

也可以定义无符号的字符类型，即unsigned char，这样取值范围就是0~255。

2.5.3 字符类型

【例2.17】字符变量的应用

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    char ch1,ch2;    //定义2个字符变量
```

```
    int n;
```

```
    ch1='A';    //字符变量ch1赋值为字符A
```

```
    printf("ch1=%c, ASCII: %d\n", ch1, ch1);
```

//分别以字符形式和整数形式输出ch1

```
    ch2='0';    //字符变量ch3赋值为字符0
```

```
    printf("ch2=%c, ASCII: %d\n", ch2, ch2);
```

```
    n='9'-'0';    //字符9的ASCII码减字符0的ASCII码，结果为整数9
```

```
    printf("n=%d\n", n);
```

```
    return 0;
```

```
}
```

输出结果:

ch1=A, ASCII: 65

ch2=0, ASCII: 48

n=9

输出字符变量
既可以用%c,
也可以用%d

2.5.3 字符类型

【例2.16】从键盘输入一个小写字母，输出它对应的大写字母。

提示：字母字符的ASCII码是连续的，大写字母'A'~'Z'对应的ASCII码是65~90，小写字母'a'~'z'对应的ASCII码是97~122，字符数据可以和整数进行计算。

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    char ch;
```

```
    scanf("%c", &ch);
```

```
    ch=ch-32;
```

```
    printf("%c", ch);
```

```
    return 0;
```

```
}
```

总结：

大写字母加32，可以得到小写字母

小写字母减32，可以得到大写字母

怎样实现小写字母
变大写字母？

2.5.3 字符类型

3. 字符数据的输出

(1) 格式化输出

例： `char ch='A';`
`printf("%c", ch);`

(2) 字符输出函数putchar

一般形式： **putchar(参数);**

功能：在显示器上输出单个字符，该函数有且仅有一个参数。

参数可以是字符型或整型的常量或变量。

例：

`char x='#';`

`putchar('A');` //参数为字符常量，输出大写字母A

`putchar(x);` //参数为字符变量，输出字符变量x的值，即输出#

`putchar('\n');` //参数为字符常量（转义字符），输出换行

`putchar(97);` //参数为整型常量，输出97对应的小写字母a

2.5.3 字符类型

4. 字符数据的输入

(1) 格式化输入

例: `char ch;`
`scanf("%c", &ch);`

(2) 字符输入函数 `getchar`

一般形式: **`getchar();`**

功能: 从键盘输入一个字符, 该函数没有参数。

如果需要使用输入的字符数据, 应该把输入的字符赋值给一个字符型变量。

例:

`char ch;`

`ch=getchar();` //输入的数据赋给字符变量ch

`putchar(ch);` //输出变量ch中的字符

`getchar();` //输入的数据不赋给任何变量

2.5.3 字符类型

5. 字符串常量

字符串常量是由一对双引号括起来的由零个或多个字符组成的字符序列。

例: "ABC", "program", "" (空串, 引号内没任何字符)

说明:

C语言中在每个字符串的末尾会自动加一个字符串结束标志'\0'
因此字符串常量在内存中占用的字节数是字符串中字符个数加1。

注意: 'k'与 "k" 是不同的

'k'是字符常量,在内存中占1个字节,

"k" 是字符串常量,在内存中占2个字节

2.5.3 字符类型

◆ 字符串常量和字符常量的区别:

(1) 字符常量用单引号括起来，字符串常量用双引号括起来。

注意：字符'A'和字符串"A"不同，'A'在内存中仅占用1个字节，而"A"则需要占用2个字节。

(2) 字符常量只能是单个字符，字符串常量可以是零个或多个字符，允许有空串。

注意：空串也需占用1个字节的存储空间。

字符'A' 占1个字节

A

字符串"A" 占2个字节

A	\0
---	----

"" 空串占1个字节

\0

2.5.4 C语言的数据类型



2.6 类型转换

2.6.1 赋值运算中的自动类型转换

【例2.18】赋值转换示例。

```
#include<stdio.h>
```

```
int main()
```

```
{    int a, b;
```

```
    float x, y;
```

```
    a=6.8/2;
```

```
    b=3.14*2*2;
```

```
    x=10/2;
```

```
    y=3.14*2*2;
```

```
    printf("a=%d, b=%d\n", a, b);
```

```
    printf("x=%f, y=%f\n", x, y);
```

```
    return 0;
```

```
}
```

输出结果:

a=3, b=12

x=5.000000, y=12.560000

2.6.1 赋值运算中的自动类型转换

1. 整型数据和实型数据之间的转换

(1) 将实型数据赋给整型变量时，截取整数部分，舍弃小数部分。

例：

```
float x=24.78;
```

```
double y=3.654321e4, z=3.654321e9;
```

```
int a, b, c;
```

```
a=x; //24.78取整后为24，赋给变量a
```

```
b=y; //y值为36543.21，取整后为36543，赋给变量b
```

```
c=z; //z值为3654321000，超过int型的取值范围，会出现数据溢出
```

```
printf("a=%d, b=%d, c=%d\n", a, b, c);
```

输出：

```
a=24, b=36543, c= -2147483648
```

2.6.1 赋值运算中的自动类型转换

1. 整型数据和实型数据之间的转换

(2) 将整型数据赋给实型变量时，数值不变，只是以浮点数形式存储到实型变量中。

例：

```
int a=245;   float x;  
x=a;        //x的值为245.0
```

(3) 将double型数据赋给float型变量时，截取前面的7位有效数字
例：

```
double x;   float m;  
x=987.333333;  
m=x;  
printf("x=%lf\n", x);  
printf("m=%f\n", m);
```

输出：

```
x=987.333333  
m=987.333313
```

2.6.1 赋值运算中的自动类型转换

2. 不同类型的整型数据间的转换（包括字符型）

(1) 将short、int型数据赋给char型变量时，内存中即将低8位的二进制序列直接原封不动传送给char型变量，高8位数据无效，且符号位的值可能发生改变，转换后的数据有可能出错。

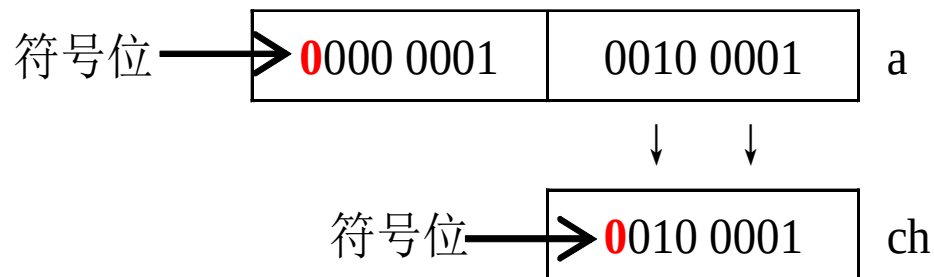
例：

```
short a=289;
```

```
char ch;
```

```
ch=a;
```

```
printf("%d", ch);
```



输出：

33

2.6.1 赋值运算中的自动类型转换

2. 不同类型的整型数据间的转换（包括字符型）

(2) 带符号的char、short型数据赋给int型变量时，需要进行符号位扩展(即原数据符号位是0，则高位补0，原数据符号位是1，则高位补1)。将无符号的数据赋给带符号的整型变量时，不存在符号位扩展的问题，只需将高位补0即可。

例：

`char ch='a';`

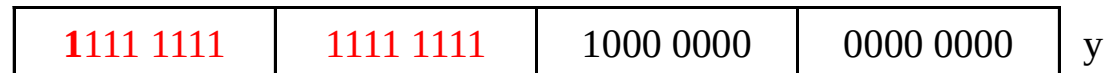
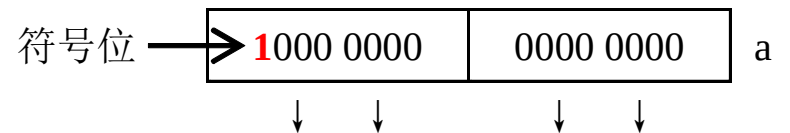
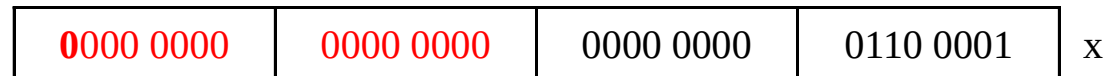
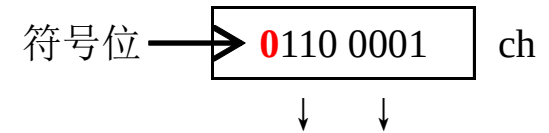
`short a=-32768;`

`int x, y;`

`x=ch;`

`y=a;`

`printf("x=%d, y=%d\n", x, y);`



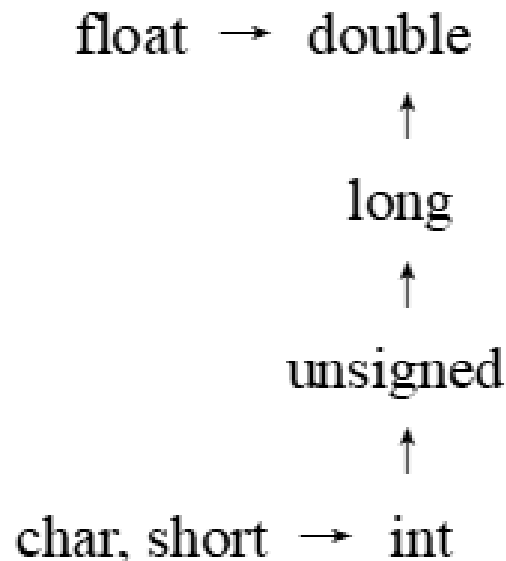
输出： x=97, y=-32768

2.6.2 表达式运算中的自动类型转换

(1) char型和short型数据参与运算时，会先转换成int型，这种转换称为**整型提升**。

(2) 类型转换按数据长度增加的方向进行，以保证数据精度不降低。

(3) 表达式中的运算对象类型不同时，先转换成同一类型，然后进行运算。



注意：横向箭头表示必然的转换，纵向箭头表示不同类型的转换方向。

◆类型转换目的：将短的数扩展为机器处理的长度，使运算符两端有相同的类型

例：计算表达式 $20+6.4$ ，因常量6.4在计算机内部是double型的，所以计算时先将20转换为double型数据20.0，再与6.4相加，表达式计算结果为double型。**(注意：int型是直接转换为double型)**

2.6.3 强制类型转换

强制类型转换是通过类型转换运算符来实现的，其功能是把表达式的运算结果强制转换成类型说明符所表示的类型。

1. 强制类型转换的一般形式

(类型说明符) (表达式)

例：

```
int a=3, b;
```

```
float x=1.5, y=7.9, z;
```

```
z=(float) a;    //将变量a的值转换为实型数据
```

```
b=(int)(x+y);  //将x+y的结果转换为整型数据
```

```
z=(int)(x)+y;  //将x转换为整型数据后与y相加，最终表达式的结果是float型
```

2.6.3 强制类型转换

2. 使用强制转换应注意的问题

(1) 类型说明符和表达式都必须加括号（单个变量可以不加）

例：

(float) (5/2) //先计算5/2，结果为2，然后将整数2转换为实数2.0

(float) 5/2 //先将整数5转换为实数5.0，再计算5.0/2，结果为2.5

(2) 无论是强制转换或是自动转换，都只是为了本次运算的需要而对变量进行的临时转换，但并不会改变变量本身的数据类型

例：

float x=3.76;

printf(" %d\n", (int)x); //将x的值3.76强制转换成整数，即取整

printf(" %f\n", x); //x本身的float类型并不改变

输出：

3

3.760000

2.7 C语言的基本标识

2.7.1 C语言字符集

(1) 字母: **A-Z, a-z**

(2) 数字: **0-9**

(3) 空白符: 空格, 制表符(跳格), 换行符的总称

(4) 标点符号、特殊字符

!	#	%	^	&	+	-	*	/	=	~	<	>	\
	.	,	;	:	?	'	“	(	)	[	]	{	}

2.7.2 标识符

标识符(identifier)是一个名字，在C语言中标识符就是常量、变量、类型、语句、标号及函数的名称。

C语言标识符有3类: 关键字、预定义标识符和用户定义标识符。

1. 关键字

已被C系统所使用的标识符称为关键字，每个关键字在C程序中都有其特定的作用，**关键字不能作为用户标识符**。

auto	break	case	char	const	continue
default	do	double	else	enum	extem
float	for	goto	if	int	long
register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void
volatile	while				

2.7.2 标识符

2. 系统预定义标识符

C语言系统提供的库函数名和编译预处理命令等构成了系统预定义标识符。

- 在程序中若使用了库文件包含，就把相应的系统预定义标识符定义在程序中了，程序设计时就可以使用这些预定义标识符。
- 如果程序中没有相应的库文件包含，用户可以定义标识符与系统预定义标识符一样的名称，但应尽量避免这样做。因为C语言系统已经规定了预定义标识符的特定含义，用户再定义与之相同的名字，便强行改变了系统原来赋予该标识符的意义，导致使用上的混淆。

注意：应尽量避免使用预定义标识符作为用户标识符。

2.7.2 标识符

3. 用户标识符

用户对程序中用到的变量、符号常量、用户函数、标号等进行的命名，称为用户标识符。

用户标识符一般满足以下规则：

- (1) 标识符只能由字母、数字和下划线3种字符组成，且第一个字符必须为字母或下划线。
- (2) 大小写敏感。

如：sum、Sum、SUM、average、_total、lotus_1_2_3

D.John、¥123、xy#z、3D64、a>b



2.7.2 标识符

3. 用户标识符

用户标识符一般满足以下规则：

- (3) ISO C没有限制标识符长度，但各个编译系统都有自己的规定和限制。
- (4) 标识符不能与“关键字”同名，最好也不与系统预先定义的“标准标识符”同名。
- (5) 在同一函数的不同复合语句中，最好不要定义相同的标识符作变量名。

◆ 定义标识符应遵循的原则

- ① 尽量做到见名知义
- ② 一般习惯上变量名、函数名用小写，而符号常量用大写
- ③ 应尽量避免使用容易认错的字符 如：数字1和小写字母l

2.7.2 标识符

4. 分隔符

包括逗号，空格，回车等，主要起分隔、间隔作用。

5. 注释符

- (1) “/*”和 “*/”构成一组注释符。编译系统将 “/* ... */”之间的所有内容看作为注释，编译时编译系统忽略注释。
- (2) 在C++中，常用双斜杠注释符 “//”，它的注释范围为当前行，如果注释分几行，在每一行的前面都需要加 “//”。

◆ 注释的作用：

- (1) 解释说明变量、语句的用途等
- (2) 在调试程序时，使用注释可以临时屏蔽一些语句

2.8 格式化输入输出函数完整版

2.8.1 格式化输出函数

一般格式：**printf(格式控制字符串, 输出项列表);**

(1) 格式控制字符串：用双引号括起来的字符串，用来指定输出数据项的类型和格式，它包括两部分：

① **普通字符**：即需要原样输出的字符，包括转义字符。

例：**printf ("Welcome! \n");**

② **格式说明**：由“%”和格式字符组成，格式说明总是由“%”开始，到格式字符终止，它的作用是将数据项按指定的格式输出。

例：**printf("x=%d, y=%d, z=%d\n", x, y, z);**

2.8.1 格式化输出函数

◆ 格式说明的完整格式:

% - 0 m.n l 或 ll 或 h 格式字符

- **%**: 表示格式说明的起始符号, 不可缺少。
- **-**: 有负号表示左对齐输出, 如省略表示右对齐输出。
- **0**: 有0表示指定空位填0, 如省略表示指定空位不填。
- **m.n**: **m**用来控制域宽, 即输出项在输出设备上所占的列数
n用来控制精度, 即控制输出的实型数据的小数位, 未指定**n**时, 默认精度为小数点后6位。

注意, 输出整数时只能指定m。

- **l**: 对整型指long型, 对实型指double型。
- **ll**: 对应long long型。
- **h**: 用于将整型格式修正为short型。

2.8.1 格式化输出函数

printf函数中常用的格式字符

格式字符 ↵	含 义 ↵
c ↵	以字符形式输出一个字符 ↵
s ↵	输出字符串 ↵
d 或 i ↵	以十进制形式输出带符号整数（正数不输出符号） ↵
u ↵	以十进制形式输出无符号整数 ↵
o ↵	以八进制形式输出无符号整数（不输出前缀 0） ↵
x(X) ↵	以十六进制形式输出无符号整数（不输出前缀 0x） ↵
f ↵	以小数形式输出 float 型数据，默认输出小数点后 6 位 ↵
e(E) ↵	以指数形式输出 float、double 型数据 ↵
g(G) ↵	自动选取 f 或 e 格式中输出宽度较短的一种格式输出 float、double 型数据 ↵

2.8.1 格式化输出函数

例:

```
float x=6.85;  
printf("x=%f\n", x);  
printf("x=%4.2f\n", x);  
printf("x=%8.4f\n", x);  
printf("x=%08.4f\n", x);  
printf("x=%-8.4f\n", x);
```

输出结果:

x=6.850000

x=6.85

x=□□6.8500

x=006.8500

x=6.8500 □□

(2) **输出项列表**: 需要输出的一些数据项, 可以是常量、变量、表达式或函数调用。

例:

```
int x=8, y=-2;  
printf("%d\n", 25);  
printf("x=%d\n", x);  
printf("x*y=%d\n", x*y);  
printf("|y|=%d\n", fabs(y));
```

输出结果:

25

x=8

x*y=-16

|y|=2

2.8.1 格式化输出函数

◆使用printf函数的注意事项

- ① 格式说明与输出项的数据类型不一致时，系统不会提示错误，但输出结果是错误的。

例：

```
int x=97;
```

```
float y=24.78;
```

```
char c=65;
```

```
printf("x=%f\n", x);
```

```
printf("y=%d\n", y);
```

```
printf("c=%c, c=%d\n", c, c);
```

输出：

x=0.000000

y=536870912

c=A,c=65

2.8.1 格式化输出函数

◆使用printf函数的注意事项

- ② 一般要求输出项列表中的每个输出项都对应一个格式说明。如果输出项的个数比格式说明的个数少，则多出来的格式说明在VS2013环境下将输出一个不确定的值（不同环境下的输出可能不同）；如果输出项个数比格式说明的个数多，则多余的输出项不会被输出。

例：

```
int x=3, y=8;  
printf(" %d,%d,%d\n", x, y);  
printf(" %d\n", x, y);
```

输出：

3, 8, 不确定值
3

- ③ 如想输出字符%，应该在格式控制字符串中用连续两个%

例： `printf(" %f%%", 1.0/3);`

输出： 0.333333%

2.8.2 格式化输入函数

一般格式: **scanf(格式控制字符串, 地址列表);**

(1) 格式控制字符串: 格式控制字符串的含义与printf类似, 它指定输入数据项的类型和格式。

格式说明的完整格式: **% * m l或ll或h 格式字符**

- *****: 用来表示跳过它对应的数据。
- **m**: 用来指定输入数据的域宽, 系统会按它自动截取数据。
(注意只有**m**, 而非**m.n**)
- **l或ll**: **l**对long型或double型, **ll**指long long整型。
- **h**: **h**用于将整型 格式修正为short型。
- **格式字符**: 与printf函数中的基本相同, 但没有**u**和**g**。

(2) 地址列表: 由若干个地址组成的列表, 可以是变量的地址或字符串的首地址, 其作用是将输入的数据存放到对应变量的存储区。

2.8.2 格式化输入函数

【例2.22】数值型数据的输入

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b, c, d;
```

```
    float x, y;
```

```
    scanf("%d%d", &a, &b);
```

```
    scanf("%2d%*2d%3d%*d", &c, &d);
```

```
    scanf("%f%4f", &x, &y);
```

```
    printf("a=%d, b=%d, c=%d,d=%d\n", a, b, c,d );
```

```
    printf("x=%f, y=%f\n", x, y);
```

```
    return 0;
```

```
}
```

输入数据格式如下：

（□表示空格，↵表示回车符）

12□256↵ //输入数据第1行

987654321↵ //输入数据第2行

2.45678↵ //输入数据第3行

4.3591↵ //输入数据第4行

程序输出：

a=12, b=256, c=98, d=543

x=2.456780, y=4.350000

2.8.2 格式化输入函数

【例2.23】字符数据的输入

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    char ch1,ch2,ch3,ch4;
```

```
    scanf("%c%c", &ch1, &ch2);
```

```
    scanf("%2c%3c", &ch3, &ch4);
```

```
    printf("ch1=%c,ch2=%c,ch3=%c,ch4=%c\n",ch1,ch2,ch3,ch4);
```

```
    return 0;
```

```
}
```

输入格式1:

abcdefg✓

输出:

ch1=a,ch2=b,ch3=c,ch4=e

输入格式2:

a□b□c□d□e□f□g✓

输出:

ch1=a,ch2=□,ch3=b,ch4=c

输入格式3:

a✓

b✓

c✓

d✓

输出:

ch1=a,ch2=
,ch3=b,ch4=c

2.8.2 格式化输入函数

(3) 使用scanf函数的注意事项

- ① 输入实型数据时不能规定小数点后的精度。

```
float x, y;
```

```
scanf("%3f%5.2f", &x, &y);
```

输入数据：123456789 ✓

输出：x=123.000000, y=-107374176.000000

- ② 如果在格式控制字符串中除了格式说明以外还有其它字符，则在输入数据时必须原样输入这些字符。

```
int a, b;
```

```
float x, y;
```

```
scanf("a=%d b=%d", &a, &b);
```

```
scanf("%f,%f", &x, &y);
```

数据输入格式如下：

a=3 b=4 ✓

1.5,6.89 ✓

2.8.2 格式化输入函数

(3) 使用scanf函数的注意事项

- ③ scanf 函数中的地址列表必须是变量的地址，即地址运算符 **&+变量名**，只写变量名编译时不会报错，但运行时会出错。

```
int a, b;
```

```
scanf("%d%d", a, b);
```

正确的写法

```
scanf("%d%d", &a, &b);
```

- ④ 输入数据未用分隔符进行分隔时，系统是根据格式字符自动获取数据，当输入数据的类型与格式字符要求不符时，就认为这一输入项结束。

```
int a; char b; float c;
```

```
scanf("%d%c%f", &a, &b, &c);
```

数据输入如下：

1234r1234.567 ✓

a=1234, b=r, c=1234.567

2.9 C语言的程序结构

2.9.1 C语句

1. 控制语句：用于控制程序的流程，9种控制语句可分成3类

if () ~ 单分支选择语句

if () ~ else ~ 双分支选择语句

switch 多分支选择语句

for () ~
while () ~
do ~ while ()

} 循环语句

continue 结束本次循环语句

break 中止语句

goto 转向语句

return 从函数返回语句

2.9.1 C语句

2. 表达式语句

表达式语句是在表达式的最后加一个分号组成

表达式语句的一般形式：**表达式;**

表达式	表达式语句	说明
a=3 （赋值表达式）	a=3; （赋值语句）	将3赋给变量a
x+y （算术表达式）	x+y; （算术表达式语句）	x+y; 是一个语句，其作用是完成 x+y 操作，是合法的，但并不将结果赋给另外的变量，所以并无实际意义
i++ （自增表达式）	i++; （自增表达式语句）	表示i的值加1

2.9.1 C语句

3. 函数调用语句：由函数调用加一个分号构成函数调用语句。

例： `printf("This is a C statement. ");` `//输出函数调用语句`

4. 空语句：只有一个分号的语句，它什么也不做。

例： `for(i=1; i<=5; i++)`

;

这个分号即空语句

5. 复合语句：用 { } 括起来的语句序列

例：

```
if(x>y)
{ int t;
  t = x;
  x = y;
  y = t;
}
```

注意：

- (1) 复合语句是一个整体, 相当于一个语句
- (2) 一个复合语句中可以包含其他复合语句
- (3) 在复合语句的花括号后不要再加分号
- (4) 复合语句中可定义变量, 但此变量只在该复合语句内有效

2.9.2 C程序结构

【例2.24】 输入两个整数，计算两者较大的数并输出。

```
#include <stdio.h>           //文件包含命令，包含C系统提供的头文件
int g_x, g_y;                //声明全局变量
int max(void );              //用户自定义函数声明
int main()                   //主函数定义
{   int c;                   //声明部分，定义变量
    scanf("%d,%d", &g_x, &g_y); //输入变量值
    c=max( );                 //调用max函数，将调用结果赋给c
    printf("max=%d",c);
    return 0;
}

int max(void )               //用户自定义函数，计算两数中较大的数
{   int z;                   //声明部分，定义变量
    if(g_x>g_y) z=g_x;        //如果g_x>g_y则将g_x的值赋给z
    else  z=g_y;              //否则 将g_y的值赋给z
    return z;                 //将z值返回，回到调用处
}
```

2.9.2 C程序结构

◆ C程序的基本结构

编译预处理命令

全局变量的定义

函数声明部分 //声明用户自己定义的函数

int main () //主函数的定义

{

声明部分 // 包括变量的声明和函数的声明

执行部分 // 主要是C语句

}

其他函数定义 // 可能有多个函数定义

{

声明部分

执行部分

}

2.9.2 C程序结构

(1) C程序由函数构成。

C语言是函数式的语言，函数是C程序的基本单位。

① 一个C源程序必须包含一个且只能一个main函数，

之外可以包含零个或多个其它函数和说明文件。

② 被调用的函数可以是系统提供的库函数，也可以是用户根据需要自己编写设计的函数。

③ C函数库非常丰富。例如，ISO C提供100多个库函数。

(2) main函数是每个程序执行的起点。

一个C程序总是从main函数开始执行的，也是从main终止的，而不论main函数在程序中的位置。

2.9.2 C程序结构

(3) 一个函数由函数首部和函数体两部分组成。

(4) C程序书写格式自由。强烈建议养成良好地书写习惯。

- 一行可以写几个语句，一个语句也可以写在多行上。
- 每条语句的最后必须有一个分号“;”表示语句的结束

(5) C语言本身不提供输入/输出语句，输入/输出操作是通过调用库函数（如scanf, printf）完成的。

(6) 预处理命令能够改进程序设计环境，提高编程效率。

2.9.3 顺序结构程序设计

【练习2.10】 输入一个三位整数，分解出它的个位、十位、百位，然后按个位、十位、百位的顺序输出。

输入：365

输出：5,6,3

【练习2.11】 懒羊羊是一只很懒的羊，比如说计算 $1234+69$ ，他只会计算 $4+9$ ，对于答案也只记录3，简单的说就是只对个位数进行加法计算，结果也只要个位数。输入两个整数，请按懒羊羊的方法输出计算结果。

输入：36 55

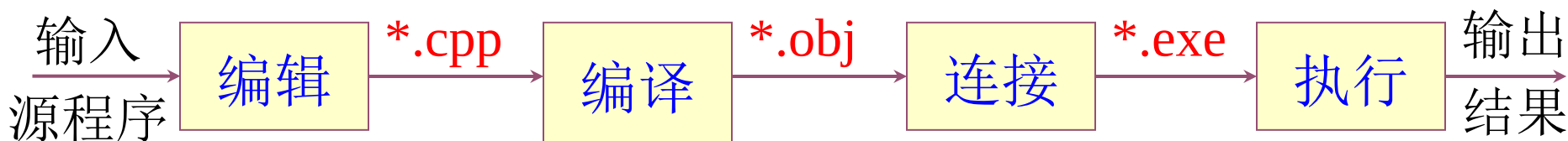
输出：1

【练习2.12】 输入两个整数 a 和 b ，计算 a/b ，求它的浮点数值，用双精度浮点类型`double`定义变量，保留小数点后8位。

输入：5 7

输出：0.71428571

2.10 编译预处理命令



- ①编辑: 将源程序输入到计算机中, 并将源程序保存在磁盘文件中
(因现在所用的多为C++环境, 所以源程序文件为*.cpp)
- ②编译: 将源程序翻译成二进制的目标代码, 同时对源程序进行语法检查, 如果有错误则修改源程序, 然后再编译, 反复该过程直到没有错误为止(注意将正确的源程序再保存一遍)
- ③连接: 将各模块的二进制目标代码与系统标准模块连接处理后, 得到一个可执行文件(*. exe文件)
- ④执行: 运行可执行文件, 检查结果是否正确, 如果有错误则应修改源程序, 再重复以上步骤, 直至程序运行正确

2.10 编译预处理命令

◆ 为了优化代码，提高编译、链接过程生成的目标代码和可执行代码的效率及适应性，C提供了编译预处理命令供程序员对编译过程中的某些环节进行选择或优化控制。

◆ 在编译过程的初期，编译器读取C源程序，首先对其中的伪指令(以#开头的指令)特殊符号进行处理，然后再进行程序语句的编译。

◆ C语言的伪指令主要包括：

- (1) 头文件包含指令
- (2) 宏定义指令
- (3) 条件编译指令

2.10.1 文件包含

- ◆ 文件包含命令的功能是把指定的“头文件”插入该命令行位置取代该命令行，从而把指定的“头文件”和当前的源程序文件连成一个源文件。

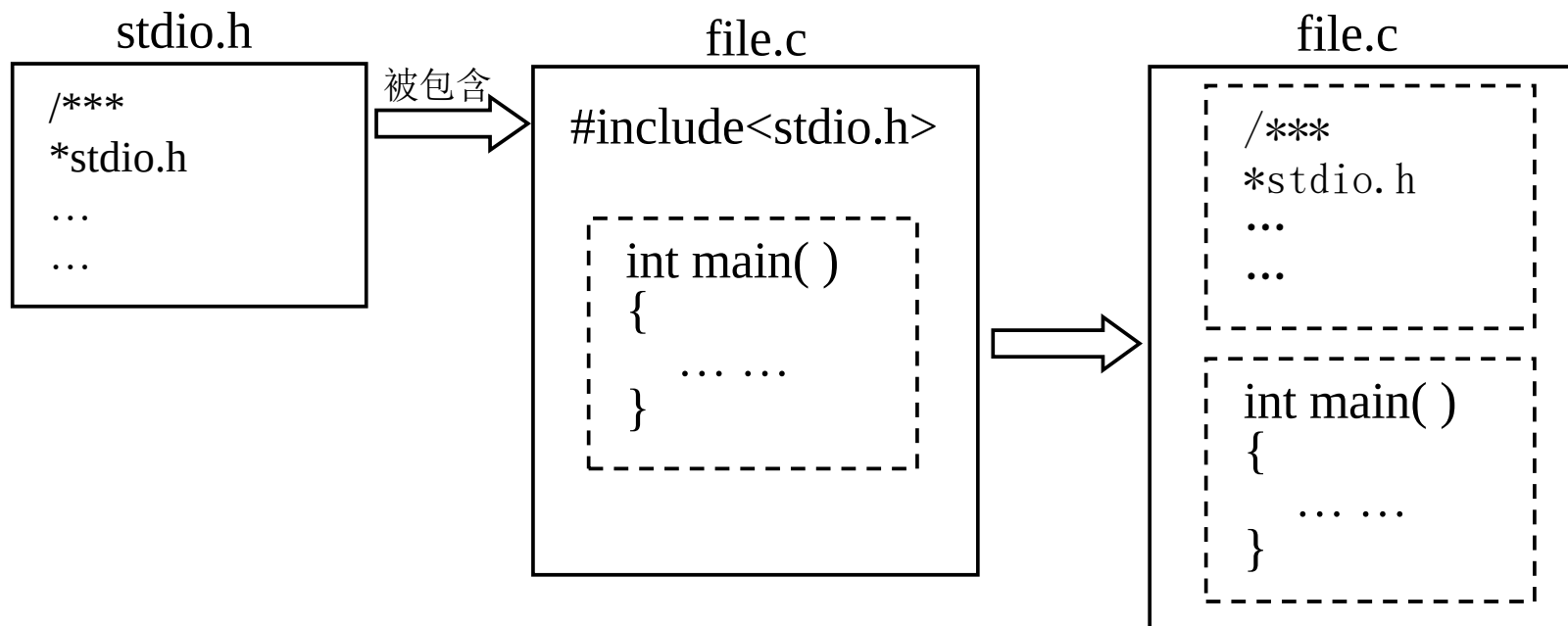


图 2.12 文件包含

2.10.1 文件包含

- ◆ 头文件是一种特殊的文件，它是共享性信息的载体。

例如，有些公用的符号常量或宏定义等可单独组成一个头文件，在其它文件的开头用包含命令包含该头文件即可使用这些符号常量或宏。这样可以避免在每个文件开头都去书写那些公用量，从而节省时间，减少出错。
- ◆ 引入头文件的目的主要是为了使这些定义可以供多个不同的C源程序使用。
- ◆ 头文件一般分为两类：
 - 标准库头文件是标准库提供的一组头文件
如：**stdio.h**和**math.h**
 - 用户自定义头文件是用户自己定义的头文件

2.10.1 文件包含

1. 文件包含命令的一般形式

(1) **#include** "头文件名"

文件标识可以包含有文件路径(即文件在哪个目录下), 使用时系统先在引用被包含文件的源文件所在的目录下寻找被包含文件, 如果找不到, 系统再按指定的标准方式检索其他目录, 直到找到为止。通常使用用户自己编写的文件时用 " "。

(2) **#include** <头文件名>

系统只按规定的标准方式检索文件目录
通常使用系统提供的标准头文件时用 <>。

2.10.1 文件包含

文件 **file1.c**

```
#include <stdio.h>
#include "file2.h"
int main ( )
{ int x , y, z ;
  scanf("%d%d", &x, &y) ;
  z = max ( x , y ) ;
  printf("max=%d\n", z) ;
  return 0;
}
```

文件 **file2.h**

```
int max (int a , int b)
{ return(a>b? a : b ) ; }
```

文件stdio.h的内容

```
int max (int a , int b)
{ return(a>b? a : b ) ; }
```

```
int main ( )
{ int x , y, z ;
  scanf("%d%d", &x, &y) ;
  z = max ( x , y ) ;
  printf("max=%d\n", z) ;
  return 0;
}
```

说明：文件file1.c和 file2.h在同一目录下

2.10.1 文件包含

2. 文件包含的特点

- (1) 一个include命令只能指定一个被包含文件。如果有多个被包含文件则需要用多个include命令, 且一个命令单独占一行。

```
#include<stdio.h>
```

```
#include<math.h>
```

```
int main( )
```

```
{
```

```
    double x, y;
```

```
    scanf(" %lf", &x);
```

```
    y=sqrt(x);
```

```
    printf("y=%lf\n", y);
```

```
    return 0;
```

```
}
```

2.10.1 文件包含

2. 文件包含的特点

(2) 文件包含可以嵌套使用

文件file1.c的内容:

```
#include<stdio.h>
#include "file2.h"
int main()
{
    double x, y, z;
    scanf("%lf%lf", &x, &y);
    z=Sumfun (x, y);
    printf("z=%.2lf\n", z);
    return 0;
}
```

文件file2.h的内容:

```
#include<math.h>
double Sumfun(double a, double b)
{
    double c;
    c=sqrt(a)+sqrt(b);
    return(c);
}
```

2.10.1 文件包含

2. 文件包含的特点

(3) 使用文件包含时, 在被包含文件中绝对不能有main函数

(4) 若文件file1.c包含file2.h, 被包含文件(file2.h)中的全局变量

文件file1.c的内容:

```
#include<stdio.h>
```

```
#include"file2.h"
```

```
int main( )
```

```
{ int x, y, z;
```

```
scanf("%d%d", &x, &y);
```

```
z=Maxfun(x, y);
```

```
printf("z=%d\n", z);
```

```
g_max=x>y?1:0; // 能使用file2.h中的全局变量g_max
```

```
printf("max=%d\n", g_max);
```

```
return 0;
```

```
}
```

文件file2.h的内容:

```
static int g_max;
```

```
int Maxfun(int a, int b)
```

```
{
```

```
g_max=a>b?a:b;
```

```
return(g_max);
```

```
}
```

2.10.2 宏定义

1. 不带参数的宏定义

格式：**#define** 标识符 字符串

宏名

宏体

注意不要忘记#

字符串不加引号

末尾没有分号

#define N 12

#define PI 3.1415926

说明：

- ① **#**表示这是一条预处理命令。
- ② 标识符为所定义的宏名，**宏名习惯上用大写字母**。
- ③ 字符串可以是**常数、表达式、格式串**等。

2.10.2 宏定义

【例2.25】 输入半径，计算圆的周长和面积。

```
1. #include<stdio.h>
2. #define PI 3.1415926    // 字符串为一个常数
3. int main( )
4. {
5.     double r, len, fArea;
6.     printf("input a radius: ");
7.     scanf("%lf", &r);
8.     len=2.0*PI*r;
9.     fArea=PI*r*r;
10.    printf("len=%.2lf,area=%.2lf\n", len, fArea);
11.    return 0;
12. }
```

预处理时宏展开后这两个赋值语句变为：

len=2.0*3.1415926*r;

fArea =3.1415926*r*r;

2.10.2 宏定义

2. 说明

(1) 宏定义不是C语句, 行末不加分号, 每条宏命令要单独占一行

```
#define N 5; ✗  
int main()  
{  
    int a = N + 2;  
    return 0;  
}
```

```
#define N 5    #define M 8 ✗  
int main()  
{  
    int x;  
    x = N + M;  
    return 0;  
}
```

2.10.2 宏定义

(2) 宏定义一般写在函数之外，其作用域为宏定义命令起到本源程序文件结束。如要终止其作用域可使用**#undef**命令。

```
#define N 5
```

```
int bbb()
```

```
{
```

```
    int c=N;
```

```
}
```

```
int main()
```

```
{
```

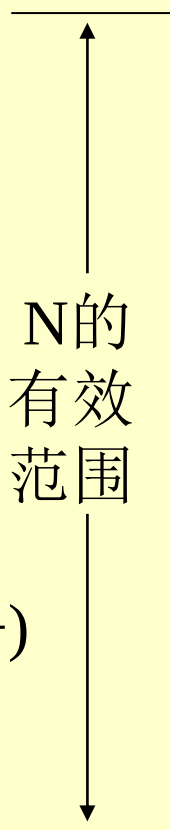
```
    int i, a=0;
```

```
    for (i=0; i<N; i++)
```

```
        a++;
```

```
    return 0;
```

```
}
```



```
#define N 5
```

```
int bbb()
```

```
{
```

```
    int c=N;
```

```
}
```

```
#undef N
```

```
int main()
```

```
{
```

```
    int i, a=0;
```

```
    for (i=0; i<N; i++)
```

```
        a++;
```

```
    return 0;
```

```
}
```



无法使用N

2.10.2 宏定义

(3) 宏定义是用宏名来表示一个字符串，在宏展开时以该字符串取代宏名，这只是一种简单的代换，字符串中可以含有任何字符，预处理程序对它不作任何检查。

如有错误，只能在编译已被宏展开后的源程序时发现。

```
#define N 2o    //将数字0输成了小写字母o
int main( )
{
    int x=5, y;
    y=x+N;      //宏展开后为 y=x+2o; 这时会出现语法错误
    printf("y=%d\n", y);
    return 0;
}
```

2.10.2 宏定义

- (4) 宏名在源程序中若用双引号括起来，则预处理程序不对其进行宏展开。

```
#define OK 100
int main( )
{
    printf("OK");           //这个OK不会进行宏展开
    printf("%d", OK);       //这个OK宏展开后为100
    return 0;
}
```

- (5) 宏定义是预处理命令的一个专用名词，它与变量定义不同，它不进行内存分配，只是作字符替换。

```
#define R 3.0    //宏定义，R不占用内存空间
float r=3.0;     //变量定义，r会占用4个字节的内存空间
```

2.10.2 宏定义

- ◆ **宏定义允许嵌套**，在宏定义的字符串中可以使用已经定义的宏名。在宏展开时由预处理程序层层替换。

【例2.26】 用宏定义的嵌套形式改写例2.25。

```
#include<stdio.h>
```

```
#define PI 3.1415926
```

```
#define R 3.0
```

```
#define L 2.0*PI*R      // PI和R是已定义的宏名
```

```
#define S PI*R*R
```

```
int main( )
```

```
{ double len, fArea;
```

```
    len=L;           //宏展开为len=2.0*3.1415926*3.0;
```

```
    fArea=S;         //宏展开为fArea=3.1415926*3.0*3.0;
```

```
    printf("len=%.2lf, area=%.2lf\n", len, fArea);
```

```
    return 0;
```

```
}
```



培养创新思维 提升编程能力